CSCE 4290 – Project 3

Twitter Sentiment Classification Using Word2Vec and Neural Networks

Student Name: Thuan Nguyen
Student ID: 11594515
Course: CSCE 4290
Project: Project 3A and Project 3B

---

1. Introduction

The objective of this project is to develop a Twitter sentiment classification model using Word2Vec embeddings and a neural network implemented with PyTorch.

The model classifies tweets into three sentiment categories:

- **Negative (0):** Tweets expressing negative opinions or emotions.
- **Neutral (1):** Tweets expressing neither clearly positive nor negative sentiment.
- **Positive (2):** Tweets expressing positive opinions or emotions.

Two experiments were conducted. Project 3A uses frozen pretrained Word2Vec embeddings, while Project 3B allows the pretrained embeddings to be updated during neural network training. The purpose is to compare how fine-tuning word embeddings affects sentiment classification performance.

2. Dataset and Preprocessing

2.1 Dataset Description

The dataset contains tweets divided into training, validation, and testing sets.

Dataset	Number of Tweets

Training	45,615
Validation	2,000
Testing	12,284
Total	**59,899**

The training dataset contains three sentiment categories.

Sentiment	Label	Number of Tweets
Negative	0	7,093
Neutral	1	20,673
Positive	2	17,849

The neutral category has the largest number of training examples, while the negative category has the smallest number.

2.2 Text Preprocessing

The following preprocessing steps were applied:

1. Converted all tweet text to lowercase.
2. Used regular expressions to tokenize tweets into words and numbers.
3. Created separate tokenized datasets for training, validation, and testing.
4. Built a vocabulary using the training tweets.
5. Added special tokens `<PAD>` and `<UNK>` for padding and unknown words.
6. Converted tweets into numerical sequences.
7. Truncated sequences longer than 50 tokens and padded shorter sequences to a maximum length of 50.

The resulting vocabulary contained 19,930 entries, including the two special tokens.

Dataset	Encoded Tensor Shape
Training	(45,615, 50)

Validation	(2,000, 50)
Testing	(12,284, 50)

3. Word2Vec Embeddings

Word2Vec was used to generate word representations from the training tweets.

The Skip-gram architecture was selected to learn relationships between words by predicting surrounding context words.

The following parameters were used:

Parameter	Value
Architecture	Skip-gram
Embedding dimension	100
Context window	5
Minimum word frequency	2
Negative samples	5
Training epochs	10
Workers	4

The resulting Word2Vec vocabulary contained 19,928 words. Two additional special tokens were added to create an embedding matrix with a shape of (19,930, 100).

4. Project 3A: Frozen Word2Vec Embeddings

4.1 Model Architecture

Project 3A uses pretrained Word2Vec embeddings with their weights frozen during classifier training.

The model consists of the following components:

1. An embedding layer initialized with pretrained Word2Vec vectors.
2. A masked mean pooling operation to calculate the average word embedding for each tweet while ignoring padding tokens.
3. A fully connected layer with 128 hidden units.
4. A ReLU activation function.
5. An output layer with three units representing the sentiment categories.

The embedding layer was configured with `freeze=True`, meaning its weights remained unchanged during classifier training.

4.2 Training Configuration

Parameter	Value
Loss function	CrossEntropyLoss
Optimizer	Adam
Learning rate	0.001
Batch size	128
Training epochs	8
Hidden dimension	128
Embeddings	Frozen

4.3 Training Results

Epoch	Training Loss	Validation Accuracy	Validation Macro F1
1	0.9106	56.25%	0.4773
2	0.8501	56.85%	0.5090
3	0.8385	57.25%	0.4846

4	0.8314	58.70%	0.5060
5	0.8258	59.25%	0.5379
6	0.8217	59.55%	0.5418
7	0.8180	59.70%	0.5490
8	0.8141	59.05%	0.5337

The training loss generally decreased throughout the eight epochs. Validation accuracy reached its highest value of 59.70% in epoch 7.

4.4 Cross-Validation Results

Stratified five-fold cross-validation was performed using three training epochs per fold.

Fold	Accuracy	Macro F1
1	59.02%	0.5157
2	59.85%	0.5053
3	57.78%	0.4598
4	58.42%	0.4952
5	58.86%	0.4965
Mean	**58.79%**	**0.4945**

The average cross-validation accuracy was 58.79%, and the average macro F1 score was 0.4945.

4.5 Test Results

The final model was evaluated using the independent test dataset.

Metric	Score

Test Accuracy	56.80%
Test Macro F1	0.5358

Classification Report

Sentiment	Precision	Recall	F1-score	Support
Negative	0.5811	0.3691	0.4514	3,972
Neutral	0.5714	0.7322	0.6419	5,937
Positive	0.5404	0.4901	0.5140	2,375
Macro Average	**0.5643**	**0.5305**	**0.5358**	**12,284**
Weighted Average	**0.5685**	**0.5680**	**0.5556**	**12,284**

The model performed best on neutral tweets, achieving an F1 score of 0.6419. Its weakest performance was on negative tweets, with an F1 score of 0.4514.

5. Project 3B: Trainable Word2Vec Embeddings

5.1 Model Architecture

Project 3B uses the same general neural network architecture as Project 3A. However, the pretrained Word2Vec embeddings are trainable.

The embedding layer was configured with `freeze=False`. Therefore, its weights were updated through backpropagation during neural network training.

The model includes:

1. A trainable embedding layer initialized with pretrained Word2Vec vectors.
2. Masked mean pooling to create a tweet representation.
3. A fully connected layer with 128 hidden units.
4. A ReLU activation function.
5. An output layer with three units for sentiment classification.

This approach allows the model to adapt the pretrained word representations to the sentiment classification task.

5.2 Training Configuration

Parameter	Value
Loss function	CrossEntropyLoss
Optimizer	Adam
Learning rate	0.001
Batch size	128
Training epochs	8
Hidden dimension	128
Embeddings	Trainable

5.3 Training Results

Epoch	Training Loss	Validation Accuracy	Validation Macro F1
1	0.8670	62.80%	0.5706
2	0.7027	65.85%	0.6268
3	0.6097	67.00%	0.6359
4	0.5410	66.80%	0.6312
5	0.4884	66.50%	0.6343
6	0.4460	66.30%	0.6333
7	0.4129	64.75%	0.6263
8	0.3832	65.60%	0.6283

Training loss decreased consistently throughout the eight epochs. Validation accuracy reached its highest value of 67.00% in epoch 3.

5.4 Cross-Validation Results

Stratified five-fold cross-validation was performed using three training epochs per fold.

Fold	Accuracy	Macro F1
1	66.09%	0.6323
2	66.49%	0.6288
3	64.55%	0.6000
4	66.07%	0.6270
5	65.17%	0.6153
Mean	**65.67%**	**0.6207**

The average cross-validation accuracy was 65.67%, and the average macro F1 score was 0.6207.

5.5 Test Results

The final trainable embedding model was evaluated using the independent test dataset.

Metric	Score
Test Accuracy	55.19%
Test Macro F1	0.5468

Classification Report

Sentiment	Precision	Recall	F1-score	Support
Negative	0.5433	0.5612	0.5521	3,972
Neutral	0.6030	0.5329	0.5658	5,937
Positive	0.4727	0.5840	0.5225	2,375

Macro Average	**0.5397**	**0.5594**	**0.5468**	**12,284**
Weighted Average	**0.5585**	**0.5519**	**0.5530**	**12,284**

The model achieved its highest class F1 score on neutral tweets at 0.5658. Its negative sentiment F1 score improved compared with Project 3A, while its positive sentiment F1 score was slightly higher as well.

6. Comparison Between Project 3A and Project 3B

The main difference between the two experiments is whether the pretrained Word2Vec embeddings are frozen or updated during classifier training.

Metric	Project 3A: Frozen	Project 3B: Trainable
Mean CV Accuracy	58.79%	**65.67%**
Mean CV Macro F1	0.4945	**0.6207**
Test Accuracy	**56.80%**	55.19%
Test Macro F1	0.5358	**0.5468**

Project 3B achieved a higher average cross-validation accuracy and macro F1 score than Project 3A. Its test macro F1 score was also slightly higher.

However, Project 3A achieved a slightly higher test accuracy, with 56.80% compared with 55.19% for Project 3B.

These results suggest that allowing Word2Vec embeddings to be fine-tuned can improve the model's ability to distinguish between sentiment categories. However, the improvement in cross-validation performance did not translate into higher overall test accuracy.

7. Discussion

The experiments demonstrate the impact of embedding fine-tuning on Twitter sentiment classification.

In Project 3A, the pretrained Word2Vec embeddings remained unchanged during classifier training. Although the classifier learned to use these representations, the word vectors could not adapt to the specific sentiment classification task.

In Project 3B, the embedding weights were updated during training. This allowed the model to adjust word representations based on the sentiment labels. The improved cross-validation results suggest that this flexibility helped the model learn more task-specific representations.

Nevertheless, Project 3B did not achieve higher test accuracy. One possible explanation is overfitting, since the training loss continued to decrease while validation performance peaked earlier and fluctuated afterward.

The dataset also contains an uneven distribution of sentiment labels. The neutral class has more training examples than the negative class, which may affect the model's ability to classify each sentiment equally well.

8. Conclusion

This project implemented two neural network models for three-class Twitter sentiment classification using pretrained Word2Vec embeddings.

Project 3A used frozen embeddings, while Project 3B allowed the embeddings to be fine-tuned during training.

Project 3B achieved better average cross-validation results and a slightly higher test macro F1 score. However, Project 3A achieved higher test accuracy.

Overall, the results show that fine-tuning pretrained word embeddings can improve sentiment classification performance, particularly when evaluating performance across multiple classes. However, higher validation and cross-validation scores do not necessarily guarantee better performance on unseen test data.

Future improvements could include adjusting the learning rate, applying dropout or other regularization methods, handling class imbalance, and experimenting with different neural network architectures.

---

9. Software and Libraries

The project was implemented using Python and the following libraries:

- PyTorch 2.15.0+cpu
- Gensim 4.4.0
- NumPy
- Pandas
- Scikit-learn
- Matplotlib

The experiments were executed using the CPU.